

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- linked sorted arrays to save a binary search
- constructing and quering layered range trees

MCS 481 Lecture 15
Computational Geometry
Jan Verschelde, 28 September 2026

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

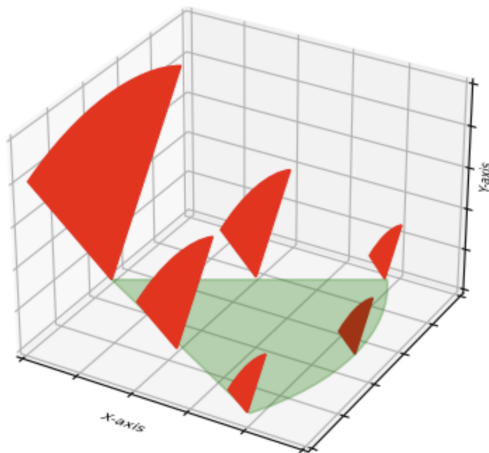
2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- linked sorted arrays to save a binary search
- constructing and querying layered range trees

a range tree is a two-level data structure



The green wedge represents the points sorted on the x-coordinate.
The red wedges represent the points sorted on the y-coordinate.

higher dimensional range trees

Generalization of a range tree from 2 to 3 dimensions:

- 1 balanced binary search tree T on the x -coordinate, and
- 2 every node v in T has as associated tree a 2D range tree.

The same type of recursion works for any dimension.

Exercise 1: Construct the three dimensional range tree for the points $P = \{(67, 63, 27), (20, 39, 58), (95, 98, 57), (17, 45, 65), (44, 24, 96), (76, 92, 78), (78, 51, 57), (25, 16, 42)\}$.

three dimensional Kd and range trees

Exercise 2: Formulate the three dimensional version of the algorithm BUILDKDTREE.

Use the example of Exercise 1 to demonstrate the algorithm.

Exercise 3: Formulate the three dimensional version of the algorithm BUILD2DRANGETREE.

Use the example of Exercise 1 to demonstrate the algorithm.

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- linked sorted arrays to save a binary search
- constructing and querying layered range trees

the cost of building a d -dimensional Kd tree

Let $T_d(n)$ be the running time of the Algorithm BUILDKD TREE for a finite set of points in dimension d .

Then $T_d(n)$ satisfies the following recurrence:

$$T_d(n) = \begin{cases} O(n \log(n)) & \text{if } d = 1, \\ O(n \log(n)) + O(\log(n))T_{d-1}(n) & \text{if } d > 1. \end{cases}$$

The two factors in $T_d(n)$ are

- 1 $O(n \log(n))$ is the cost to build a balanced binary search tree for the first coordinate x_1 ;
- 2 $O(\log(n))$ bounds the depth of a balanced tree of n elements.

$T_d(n)$ resolves to $O(n \log^{d-1}(n))$.

The same bound holds for the storage.

the cost of searching a d -dimensional Kd tree

Let $Q_d(n)$ denote the cost to query a d -dimensional range tree.

Then $Q_d(n)$ satisfies the following recurrence:

$$Q_d(n) = \begin{cases} O(\log(n)) & \text{if } d = 1, \\ O(\log(n)) + O(\log(n))Q_{d-1}(n) & \text{if } d > 1. \end{cases}$$

This recurrence resolves to $O(\log^d(n))$.

Exercise 4: Resolve the recurrence for $Q_d(n)$.

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- linked sorted arrays to save a binary search
- constructing and querying layered range trees

equal coordinates in database queries

An example of a database query: list all employees within a given salary range and a number of children between 2 and 5.

- 1 In our treatment with geometric data structures, we assumed all points have distinct coordinates, that is: no two points have the same x - or y -coordinate.
- 2 This is an unrealistic assumption, in particular with the number of children.

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- linked sorted arrays to save a binary search
- constructing and querying layered range trees

composite numbers

For simplicity we assumed no two points had the same coordinates. We remove this restriction with composite numbers.

Definition (composite number)

For two real numbers a and b , *the composite number* is $(a|b)$.

We compare two composite numbers as follows:

$$(a|b) < (a'|b') \iff (a < a') \text{ or } (a = a' \text{ and } b < b'),$$

which corresponds to a lexicographic order.

Application: replace $p = (p_x, p_y)$ by a pair of composite numbers $\hat{p} = ((p_x|p_y), (p_y|p_x))$.

For all $p, q \in P$, construct $\hat{P}$, then $\hat{p}, \hat{q}$ have distinct coordinates.

a numerical example

Consider $P = \{(0, 0), (1, 0), (0, 1), (1, 1)\}$.

With composite numbers, the set P becomes

$$\hat{P} = \{((0|0), (0|0)), ((1|0), (0|1)), ((0|1), (1|0)), ((1|1), (1|1))\}.$$

All points in $\hat{P}$ has distinct coordinates:

- x -coordinates: $(0|0), (1|0), (0|1), (1|1)$;
- y -coordinates: $(0|0), (0|1), (1|0), (1|1)$.

range query with composite numbers

Points $p = (p_x, p_y)$ are replaced by $\hat{p} = ((p_x|p_y), (p_y|p_x))$.

The range query $R = [x : x'] \times [y : y']$ is replaced by

$$\hat{R} = [(x| - \infty) : (x'| + \infty)] \times [(y| - \infty) : (y'| + \infty)].$$

Lemma (equivalent membership)

For any point p and query range R : $p \in R \Leftrightarrow \hat{p} \in \hat{R}$.

proof of equivalent membership lemma

$$\hat{R} = [(x| - \infty) : (x'| + \infty)] \times [(y| - \infty) : (y'| + \infty)].$$

Lemma (equivalent membership)

For any point p and query range R : $p \in R \Leftrightarrow \hat{p} \in \hat{R}$.

Proof. $p = (p_x, p_y) \in [x : x'] \times [y : y'] \Leftrightarrow x \leq p_x < x'$ and $y \leq p_y < y'$.

$$\hat{p} = ((p_x|p_y), (p_y|p_x)) \in [(x| - \infty) : (x'| + \infty)] \times [(y| - \infty) : (y'| + \infty)]$$

$$\Leftrightarrow (x| - \infty) \leq (p_x|p_y) < (x'| + \infty) \text{ and} \\ (y| - \infty) \leq (p_y|p_x) < (y'| + \infty)$$

$$\Leftrightarrow x \leq p_x \text{ and } p_x < x', \text{ because } -\infty < p_y < +\infty, \\ y \leq p_y \text{ and } p_y < y', \text{ because } -\infty < p_x < +\infty$$

$$\Leftrightarrow p \in [x : x'] \times [y : y'].$$

Q.E.D.

composite numbers in three dimensions

Exercise 5:

Describe the application of composite numbers to points in dimension three.

As example, consider the points that span a cube.

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- **linked sorted arrays to save a binary search**
- constructing and querying layered range trees

fractional cascading

The search in a two dimensional range tree costs $O(\log^2(n) + k)$, for a set of n points, with output size k .

With a *layered range tree*, the binary search for y is avoided via the use of extra pointers, reduced to $O(\log(n) + k)$.

Why is this possible?

The main point is that we search *with the same query range*.

nine points in the plane

Let $P = \{ (11, 84), (20, 54), (24, 38), (29, 85), (41, 85),$
 $(48, 40), (61, 72), (85, 4), (92, 99) \}.$

Then we have two lists of sorted coordinates:

- $X = [11, 20, 24, 29, 41, 48, 61, 85, 92],$ and
- $Y = [4, 38, 40, 54, 72, 84, 85, 85, 99].$

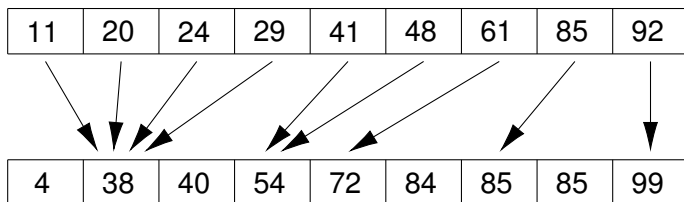
The associated tree uses Y in the balanced binary search tree.

linked sorted arrays

Let $P = \{ (11, 84), (20, 54), (24, 38), (29, 85), (41, 85), (48, 40), (61, 72), (85, 4), (92, 99) \}$.

$X = [11, 20, 24, 29, 41, 48, 61, 85, 92]$, sorted x -coordinates

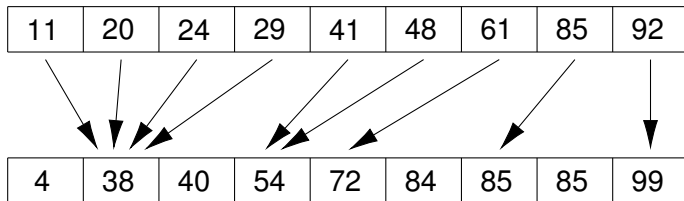
$Y = [4, 38, 40, 54, 72, 84, 85, 85, 99]$, sorted y -coordinates



Every element x in X has a pointer to the first element in Y which is either equal to x or larger than x .

saving a binary search in Y

Every element x in X has a pointer to the first element in Y which is either equal to x or larger than x .



Consider a query interval $[25 : 60]$:

- 1 A binary search in the first array finds 29, the smallest > 25 .
- 2 We walk in the first array till the first key larger than 60.
- 3 In the second array we follow the pointer from 29 to 38.
- 4 We walk in the second array till the first key larger than 60.

Higher Dimensions and General Points

1 Higher Dimensional Range Trees

- recursive multi-level data structures
- cost of building and searching

2 General Points

- equal coordinates in database queries
- composite numbers for equal coordinates

3 Fractional Cascading

- linked sorted arrays to save a binary search
- constructing and querying layered range trees

constructing a layered range tree

Let $P(v)$ be the canonical subset of the node v in a range tree, which collects in its leaves all points in the subtree under v . Observe:

- $P(\text{LEFTCHILD}(v)) \subseteq P(v)$,
- $P(\text{RIGHTCHILD}(v)) \subseteq P(v)$.

The data in $P(v)$ is stored in an array $A(v)$ with pointers to

- $A(\text{LEFTCHILD}(v))$ stores $P(\text{LEFTCHILD}(v))$,
- $A(\text{RIGHTCHILD}(v))$ stores $P(\text{RIGHTCHILD}(v))$.

For a point p with y -coordinate p_y stored at v , we store pointers

- to p' in $A(\text{LEFTCHILD}(v))$, such that p'_y is the smallest one $\geq p_y$,
- to p' in $A(\text{RIGHTCHILD}(v))$, such that p'_y is the smallest one $\geq p_y$.

This modification of a range tree is *a layered range tree*.

querying a layered range tree

Querying a layered range tree with $[x : x'] \times [y : y']$ goes as follows:

- 1 Use $[x : x']$ to compute v_{split} in $O(\log(n))$ time.
- 2 In $A(v_{\text{split}})$, find entry with smallest y -coordinate $\geq y$, which can be done in $O(\log(n))$ time.
- 3 While searching with x and x' in the main tree, in associated arrays, keep track of entries with smallest y -coordinate $\geq y$.
- 4 From where the point is with the smallest y -coordinate $\geq y$, we walk through the arrays and report points in $[y : y']$.

The query time is then $O(\log(n) + k)$ where k is the output size.

summary and exercises

We ended chapter 5 in the textbook.

Consider the following activities, listed below.

- 1 Write the solutions to exercises 1, 2, 3, 4, and 5.
- 2 Consult the CGAL documentation and example code for working with Kd and range trees.
- 3 Consider the exercises 10, 11, 12 in the textbook.

a summary of the first five chapters

what we covered in the first five weeks of the semester

- 1 The convex hull of n points in the plane has a $O(n \log(n))$ cost.
- 2 A sweep line algorithm computes the I intersection points of n line segments in time $O((n + I) \log(n))$.
Maps are represented by doubly connected edge lists.
- 3 A simple polygon with n vertices can be partitioned into y -monotone polygons in $O(n \log(n))$ time using $O(n)$ storage.
- 4 A randomized algorithm solves the two dimensional linear programming problem with n inequalities in $O(n)$ time.
- 5 Kd trees reduce the cost of database queries to $O(\sqrt{n} + k)$, for a data set of size n and where k is the output size.
Range trees are multi-level data structures to speedup queries.